Experiment 3

Aim :

This lab foucuses on the compilation of the program using gdb compiler and helps in understand the core concept of Computer architecture and also the address indexing in the system this experiment helps us in understanding the following points.

Basic GDB commands
Compiling C code for Stack, Queue, Linked List using gdb

Procedure :

Step 1: Write separate C programs for implementing the following data structures:

1. **Stack**

Implement using array or linked list
Perform operations:

- Push
- Pop
- Peek

Observation:

Stack follows LIFO (Last In First Out) principle and is stored in stack memory.

```
#include <stdio.h>
#define SIZE 5
int stack[SIZE];
int top = -1;
void push(int value) {
    if (top == SIZE - 1) {
        printf("Overflow\n");
        return;
    }
    stack[++top] = value;
}
void pop() {
    if (top == -1) {
        printf("Underflow\n");
        return;
    }
    top--;
}
int main() {
    push(10);
    push(20);
    pop();
    return 0;
}
```

Stack Code

2. Queue

Implement using array
Perform operations:

- Enqueue
- Dequeue

Observation:

Queue follows **FIFO (First In First Out)** and uses **indexed addressing**.

```c
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = value;
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Underflow\n");
        return;
    }
    front++;
}

int main() {
    enqueue(1);
    enqueue(2);
    dequeue();
    return 0;
}
```

Queue Code

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void insert(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    insert(10);
    insert(20);
    display();
    return 0;
}
```

Linked List Code

Linked List

Use dynamic memory allocation (malloc)
Perform operations:

- Insertion
- Deletion
- Traversal

Observation:
Linked list uses **heap memory** and **pointer-based addressing**.

Output after running on gbd compiler (means after executing the following commands)

gdb ./stack
break main
run
next
step # go inside push()
print top
print stack
continue

```
Breakpoint 2, push (value=10) at stackds.c:8
8           if (top == SIZE - 1) {
(gdb) n
12          top++;
(gdb) n

Breakpoint 1, push (value=10) at stackds.c:13
13          stack[top] = value;   // <-- Put breakpoint here
(gdb) n
14      }
(gdb) n
main () at stackds.c:26
26              (20);
(gdb) n

Breakpoint 2, push (value=20) at stackds.c:8
8           if (top == SIZE - 1) {
(gdb) n
12          top++;
(gdb) n

Breakpoint 1, push (value=20) at stackds.c:13
13          stack[top] = value;   // <-- Put breakpoint here
(gdb) n
14      }
(gdb) p top
$1 = 1
(gdb) p stack
$2 = {10, 20, 0, 0, 0}
(gdb) p stack[0]
$3 = 10
(gdb) p stack[1]
$4 = 20
(gdb) print &stack
$5 = (int (*)[5]) 0x555555558030 <stack>
(gdb) p &stack[1]
$6 = (int *) 0x555555558034 <stack+4>
(gdb) p &stack[0]
$7 = (int *) 0x555555558030 <stack>
```

gdb ./queue
break main
run
step # enter enqueue
print front
print rear
print queue
continue

gdb ./linkedlist
break main
run
step # go inside insert
print head
print *head
next
continue

```
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ nano queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gcc -std=c99 -g -o test queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gdb ./test
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test...
(gdb) ^[[200~b enqueue
Undefined command: "".  Try "help".
(gdb) b enqueue
Breakpoint 1 at 0x1190: file queue.c, line 10.
(gdb) run
Starting program: /home/nitdelhi/test

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, enqueue (value=10) at queue.c:10
10          if (rear == SIZE - 1) {
(gdb) n
15          if (front == -1)
(gdb) n
16              front = 0;
(gdb) n
18          rear++;
```

```
16              front = 0;
(gdb) n
18          rear++;
(gdb) n
19          queue[rear] = value;   // <-- Breakpoint can be set here
(gdb) n
20                ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 10
21      }
(gdb) n
main () at queue.c:51
51                (20);  // 2nd value
(gdb) n

Breakpoint 1, enqueue (value=20) at queue.c:10
10          if (rear == SIZE - 1) {
(gdb) n
15          if (front == -1)
(gdb) n
18          rear++;
(gdb) n
19          queue[rear] = value;   // <-- Breakpoint can be set here
(gdb) n
20                ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 20
21      }
(gdb) n
main () at queue.c:52
52                (30);  // 3rd value
(gdb) n

Breakpoint 1, enqueue (value=30) at queue.c:10
10          if (rear == SIZE - 1) {
(gdb) n
15          if (front == -1)
(gdb) n
18          rear++;
(gdb) n
19          queue[rear] = value;   // <-- Breakpoint can be set here
(gdb) n
20                ("Enqueued: %d\n", value);
(gdb) p queue
```

```
19          queue[rear] = value;   // <-- Breakpoint can be set here
(gdb) n
20                ("Enqueued: %d\n", value);
(gdb) p queue
$1 = {10, 20, 30, 0, 0}
(gdb)
$2 = {10, 20, 30, 0, 0}
(gdb) p &queue[0]
$3 = (int *) 0x555555558030 <queue>
(gdb) p &queue[1]
$4 = (int *) 0x555555558034 <queue+4>
(gdb) p &queue[2]
$5 = (int *) 0x555555558038 <queue+8>
(gdb)
```

Result

-> Successfully compiled and debugged C programs using the **GDB**.
-> Learned and applied **basic debugging commands**.
-> Understood memory behaviour of:
- Stack (LIFO, stack memory)
- Queue (FIFO, indexed memory)
- Linked List (dynamic memory, pointers)

-> Observed how **address indexing works internally**.
-> Gained practical understanding of:
- Memory layout
- Pointer operations
- Function call stack